

Récupération de fichiers dans un espace de stockage

Transition, transformation, conversion

54616

Dans la mémoire de nos appareils:

1

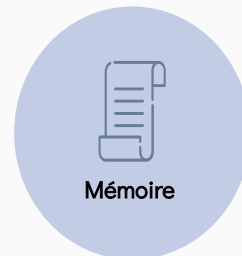
Table d'allocation

gestion, abstraction,
protection, MMU

2

Stockage mémoire

Mémoire réelle,
RAM ou disque



Dans la mémoire de nos appareils:



Dans la mémoire de nos appareils:

1

Table d'allocation
gestion, abstraction,
protection, MMU

Id \ donnés	Nom	Extension	Octet Début/Fin
1	image1	jpeg	0-112
2	image2	png	113-438
3	video1	mov	439-504
⋮	⋮	⋮	⋮

```

FFD8FFE4649460000A
251D0E1B10101B104A4
6494600A2D0E1B10101
B51010...0010FFD989
504E47400090607A464
94601D0E1B10101B213
12151121310906071D0
E12131313...151717A
464946010191A2D0E1B
10101B2131215112131
090651D1E149454E44A
E426082000001B1A181
81B1A1818171717181D
181D2820191A251D181
821312125292B2E2E00
0001B78332C3728...
  
```

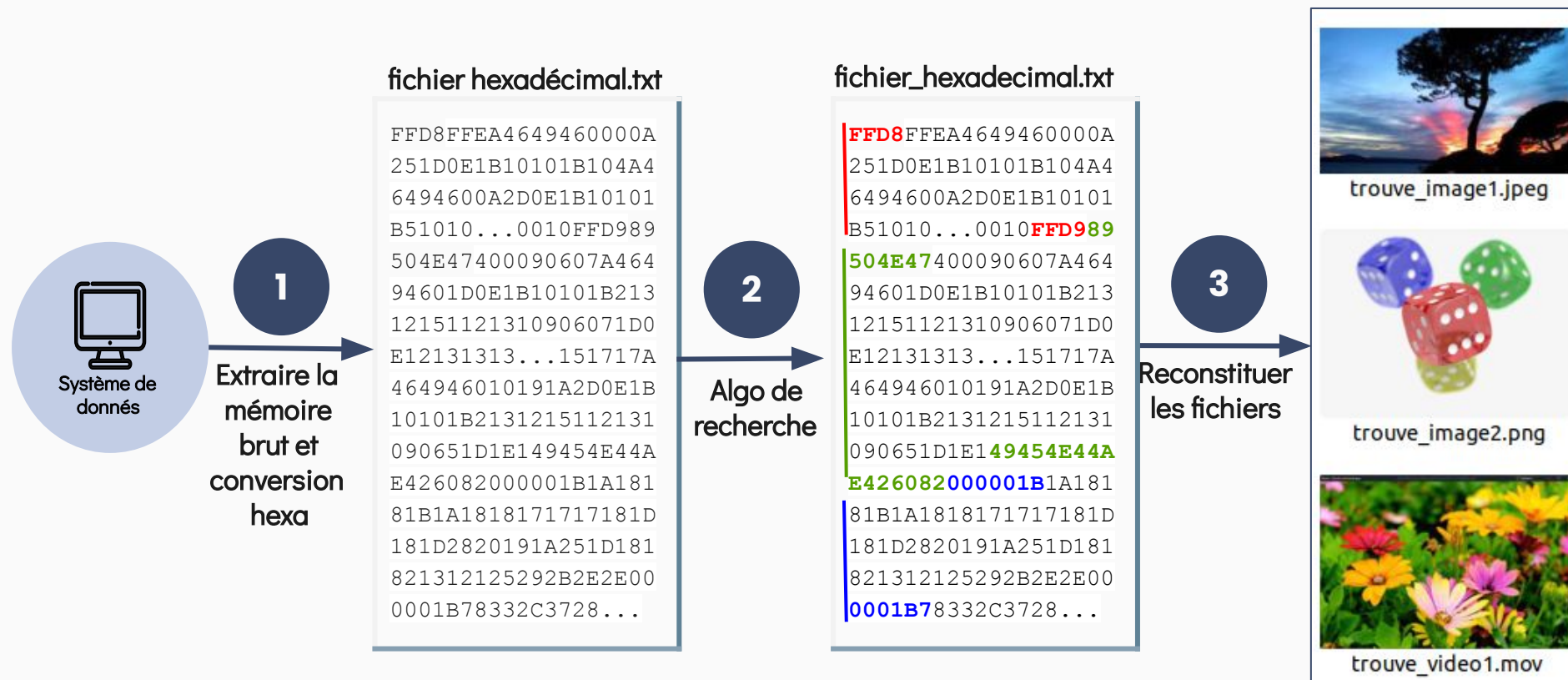
2

Stockage mémoire
Mémoire réelle,
RAM ou disque



S'il n'y a pas de table d'allocation ?

Le principe de la récupération:



La récupération des fichiers supposément perdus à partir des **données brutes** d'un espace de stockage est-elle **théoriquement possible** et aussi **réalisable en pratique ?**

01

L'accès à la mémoire

02

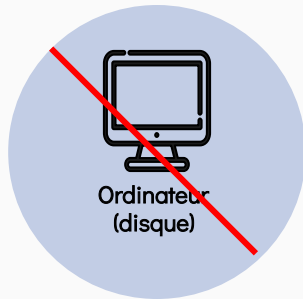
Simulation d'un système de stockage

03

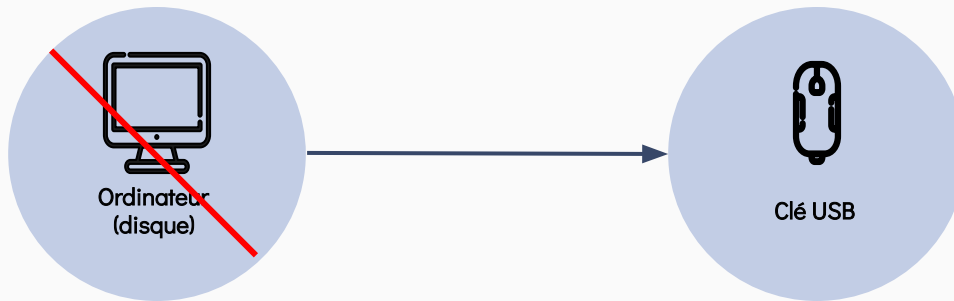
Récupérer l'arborescence de fichiers



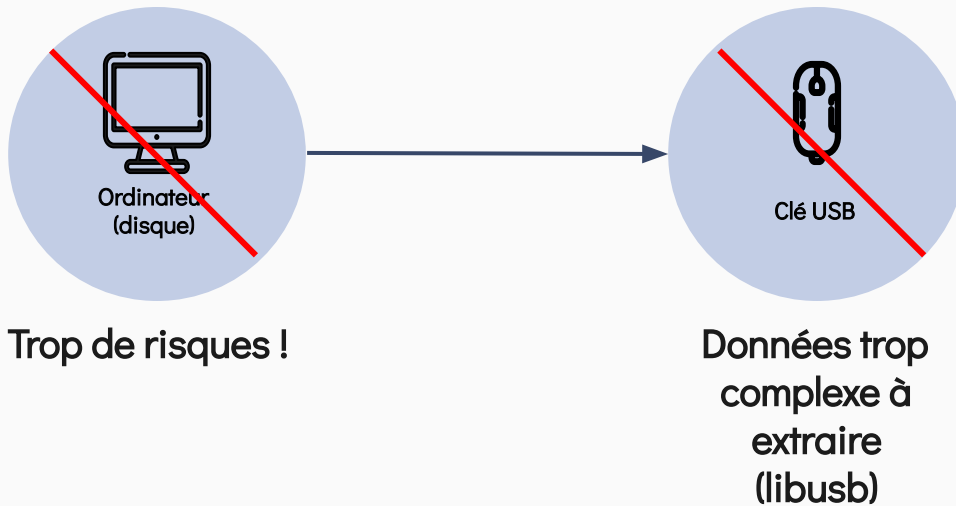
Ordinateur
(disque)

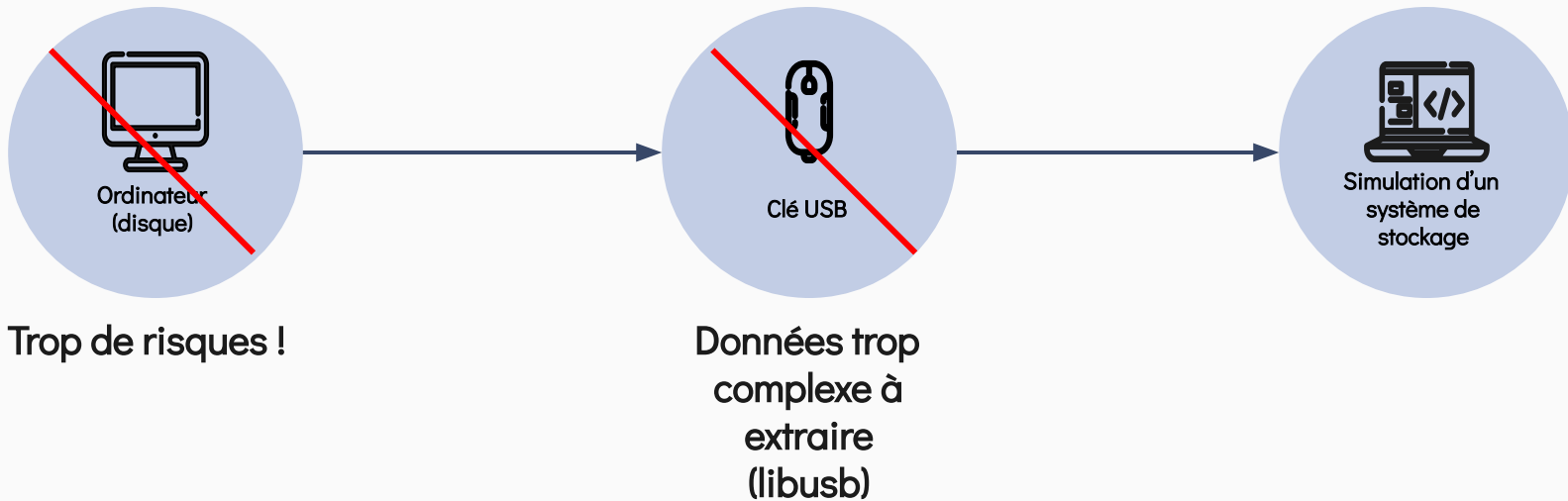


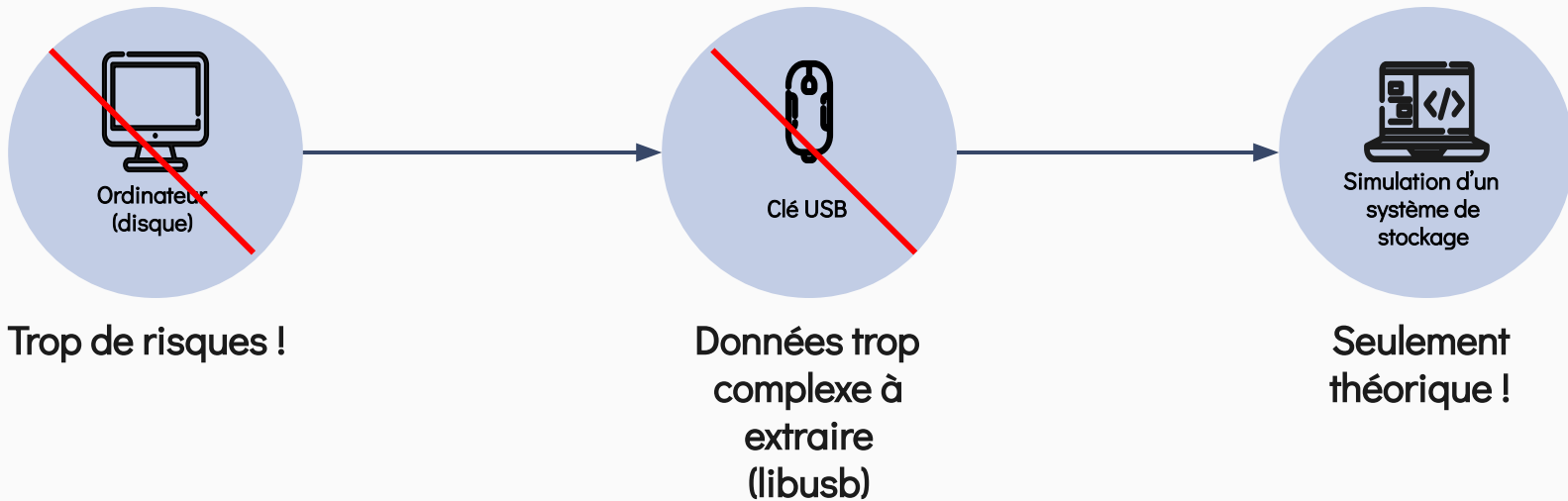
Trop de risques !



Trop de risques !







2: Simulation d'un système de stockage

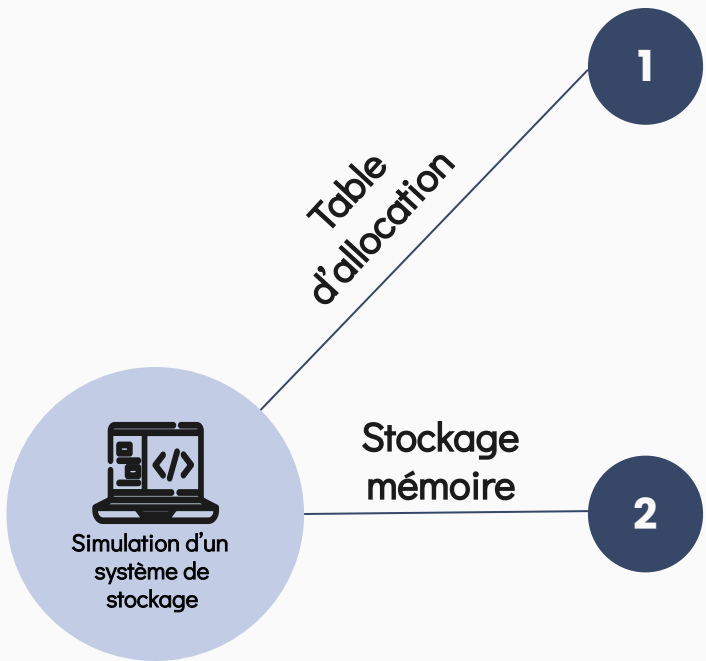
Table
d'allocation

1

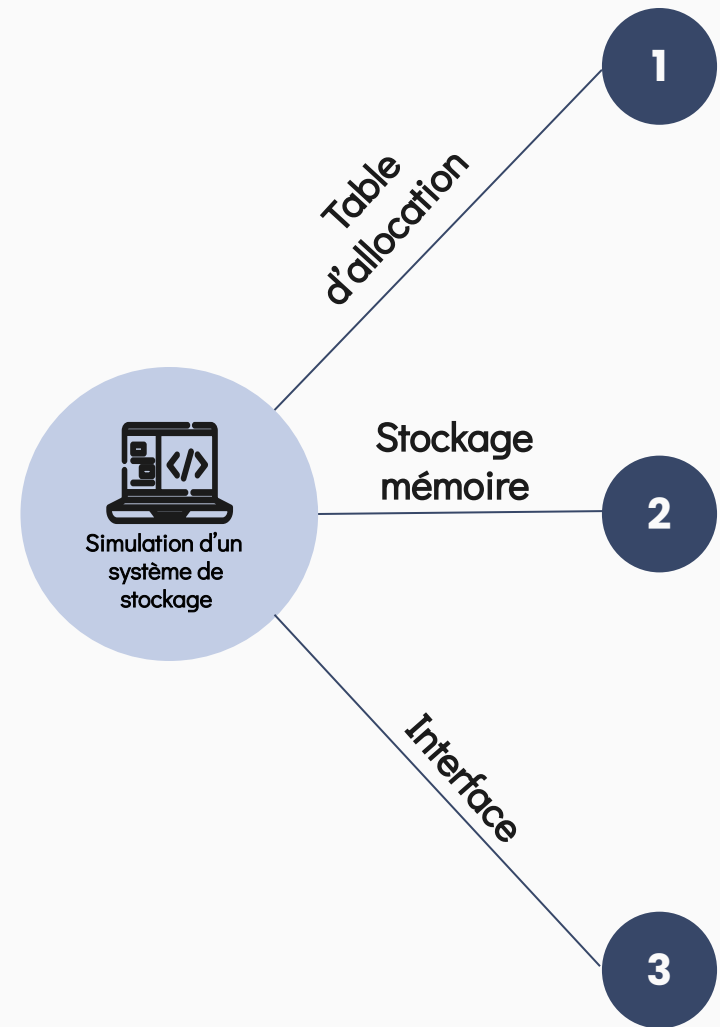


Simulation d'un
système de
stockage

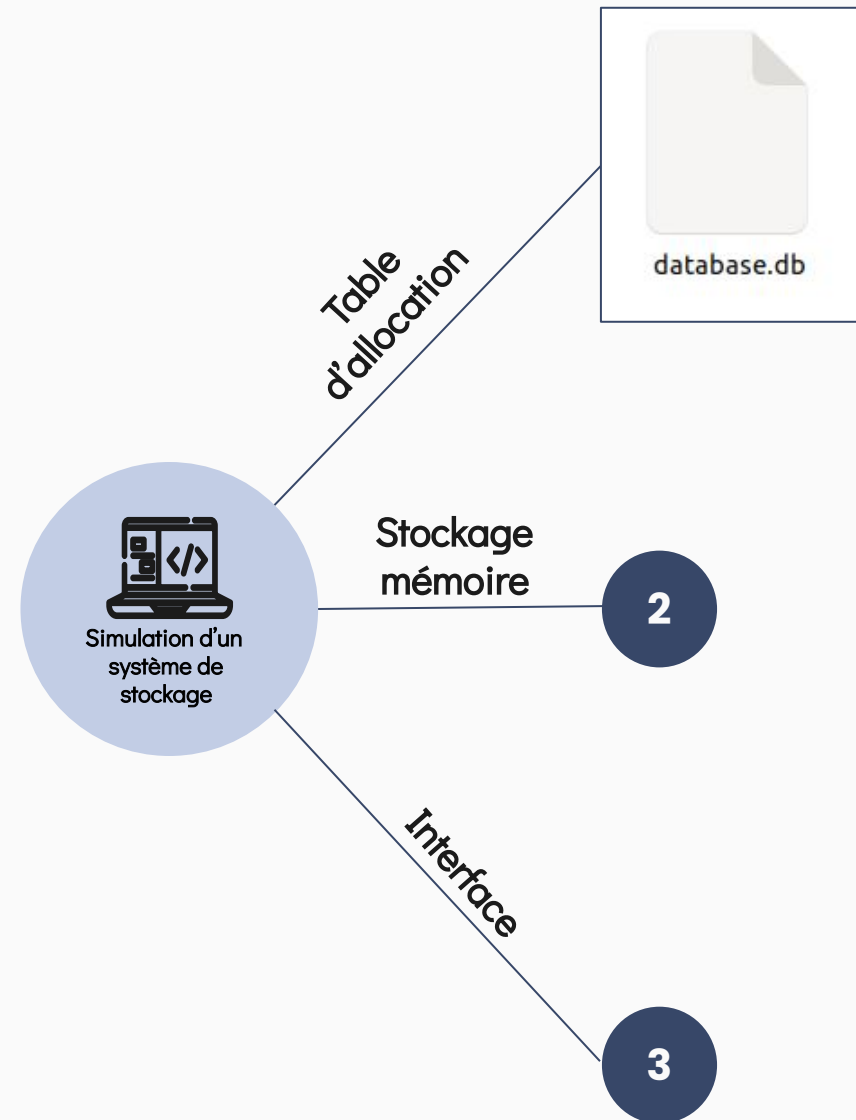
2: Simulation d'un système de stockage



2: Simulation d'un système de stockage



2.1 Création et utilisation de la base



2.1 Création et utilisation de la base

```
523 int main() {
524     sqlite3 *database;
525     char *message_erreur = 0;
526     int resultat;
527     resultat = sqlite3_open("database.db", &database);
528     if (resultat) {
529         fprintf(stderr, "Erreur d'ouverture de la base de données : %s\n", sqlite3_errmsg(database));
530         return resultat;
531     }
532     const char *requete_creation_table =
533         "CREATE TABLE IF NOT EXISTS fichiers("
534         "id INTEGER PRIMARY KEY AUTOINCREMENT, "
535         "extension TEXT NOT NULL, "
536         "nom TEXT NOT NULL, "
537         "adresse TEXT NOT NULL);";
538
539     resultat = sqlite3_exec(database, requete_creation_table, 0, 0, &message_erreur);
540     if (resultat != SQLITE_OK) {
541         fprintf(stderr, "Erreur de création de table : %s\n", message_erreur);
542         sqlite3_free(message_erreur);
543         sqlite3_close(database);
544         return resultat;
545     } else {
546         fprintf(stdout, "Table 'fichiers' créée ou déjà existante.\n");
547     }
548
549     interface(database);
550
551     sqlite3_close(database);
552
553     return 0;
554 }
```

2.1 Création et utilisation de la base

```
523 int main() {
524     sqlite3 *database;
525     char *message_erreur = 0;
526     int resultat;
527     resultat = sqlite3_open "database.db", &database);
528     if (resultat) {
529         fprintf(stderr, "Erreur d'ouverture de la base de données : %s\n", sqlite3_errmsg database));
530         return resultat;
531     }
532     const char *requete_creation_table =
533         "CREATE TABLE IF NOT EXISTS fichiers("
534         "id INTEGER PRIMARY KEY AUTOINCREMENT, "
535         "extension TEXT NOT NULL, "
536         "nom TEXT NOT NULL, "
537         "adresse TEXT NOT NULL);";
538
539     resultat = sqlite3_exec database, requete_creation_table, 0, 0, &message_erreur);
540     if (resultat != SQLITE_OK) {
541         fprintf(stderr, "Erreur de création de table : %s\n", message_erreur);
542         sqlite3_free message_erreur);
543         sqlite3_close(database);
544         return resultat;
545     } else {
546         fprintf(stdout, "Table 'fichiers' créée ou déjà existante.\n");
547     }
548
549     interface(database);
550
551     sqlite3_close(database);
552
553     return 0;
554 }
```

On utilise SQLite3

2.1 Création et utilisation de la base

```
532  const char *requete_creation_table =  
533      "CREATE TABLE IF NOT EXISTS fichiers("  
534      "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535      "extension TEXT NOT NULL,"  
536      "nom TEXT NOT NULL,"  
537      "adresse TEXT NOT NULL);";  
538
```

2.1 Création et utilisation de la base

```
532  const char *requete_creation_table =  
533      "CREATE TABLE IF NOT EXISTS fichiers("  
534      "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535      "extension TEXT NOT NULL,"  
536      "nom TEXT NOT NULL,"  
537      "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20  typedef struct {  
21      int id;  
22      char nom[128];  
23      char extension[16];  
24      char adresse[256];  
25      long offset;  
26      long longueur;  
27  } ftype;
```

2.1 Création et utilisation de la base

```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL, "  
536     "nom TEXT NOT NULL, "  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

2.1 Création et utilisation de la base

```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL,"  
536     "nom TEXT NOT NULL,"  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

2.1 Création et utilisation de la base

```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL, "  
536     "nom TEXT NOT NULL, "  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

2.1 Création et utilisation de la base

```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL, "  
536     "nom TEXT NOT NULL, "  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

2.1 Création et utilisation de la base

```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL, "  
536     "nom TEXT NOT NULL, "  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

2.1 Création et utilisation de la base

```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL,"  
536     "nom TEXT NOT NULL,"  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

2.1 Création et utilisation de la base

```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL, "  
536     "nom TEXT NOT NULL, "  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

2.1 Création et utilisation de la base

```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL, "  
536     "nom TEXT NOT NULL, "  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

2.1 Création et utilisation de la base

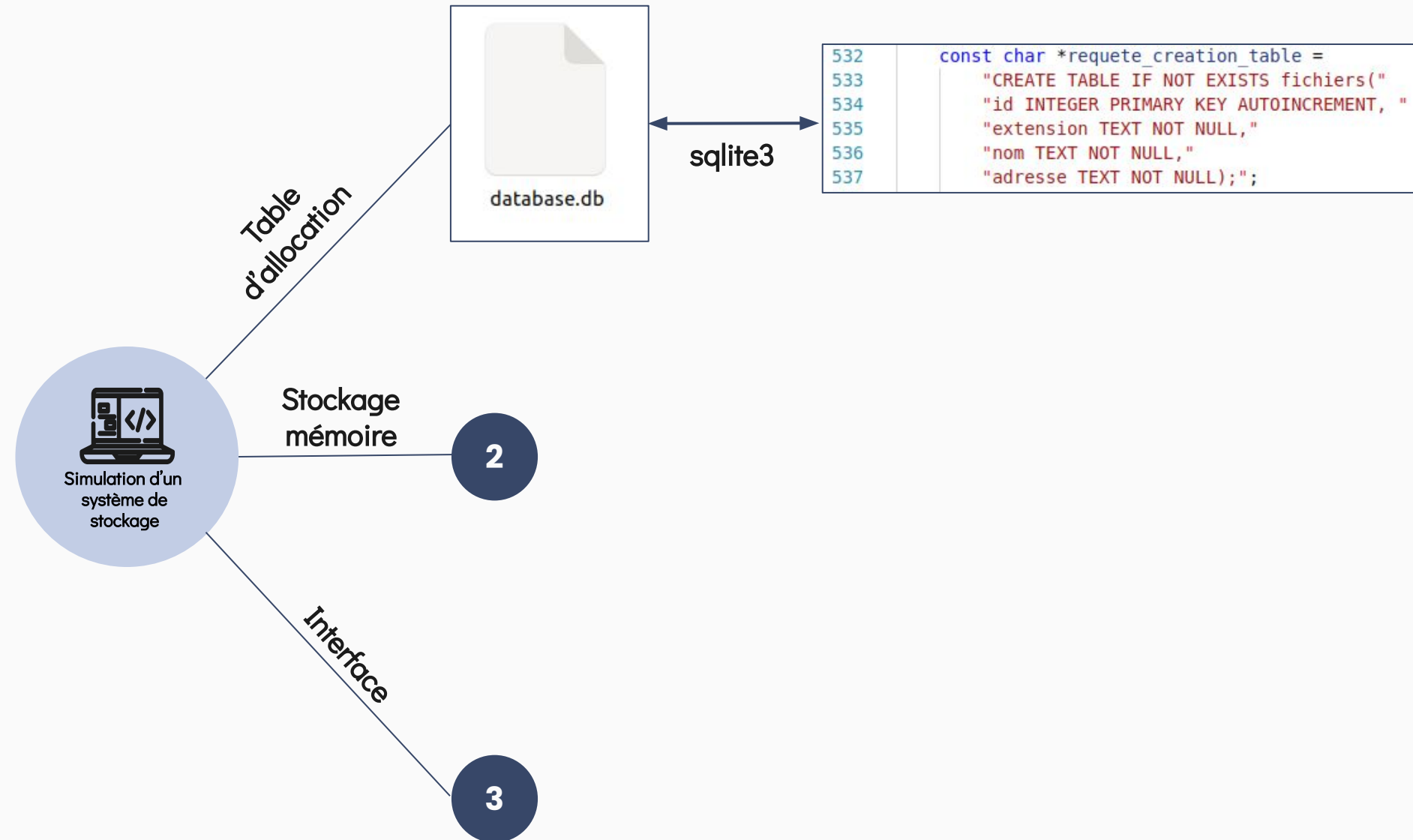
```
532 const char *requete_creation_table =  
533     "CREATE TABLE IF NOT EXISTS fichiers("  
534     "id INTEGER PRIMARY KEY AUTOINCREMENT, "  
535     "extension TEXT NOT NULL, "  
536     "nom TEXT NOT NULL, "  
537     "adresse TEXT NOT NULL);";  
538
```

sqlite3

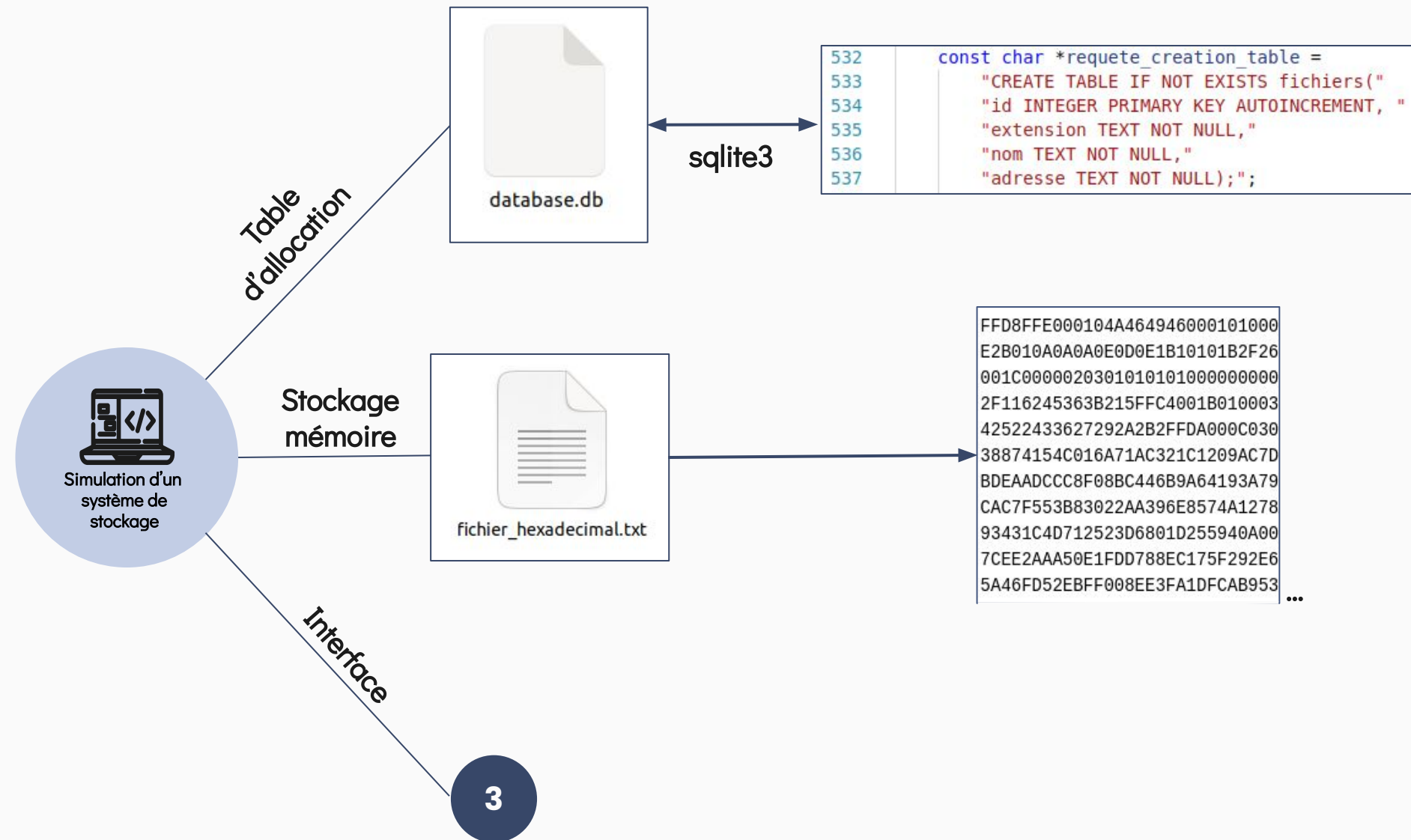
```
20 typedef struct {  
21     int id;  
22     char nom[128];  
23     char extension[16];  
24     char adresse[256];  
25     long offset;  
26     long longueur;  
27 } ftype;
```

```
29 void nom_data(sqlite3 *db, ftype *fichier) {  
30     char requete[512];  
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);  
32  
33     sqlite3_stmt *stmt;  
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {  
35         if (sqlite3_step(stmt) == SQLITE_ROW) {  
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);  
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));  
38         }  
39     }  
40     sqlite3_finalize(stmt);  
41 }  
42  
43 > void extension_data(sqlite3 *db, ftype *fichier) { ...  
56  
57 > void adresse_data(sqlite3 *db, ftype *fichier) { ...  
70  
71 > void offset_data(sqlite3 *db, ftype *fichier) { ...  
83  
84 > void longueur_data(sqlite3 *db, ftype *fichier) { ...
```

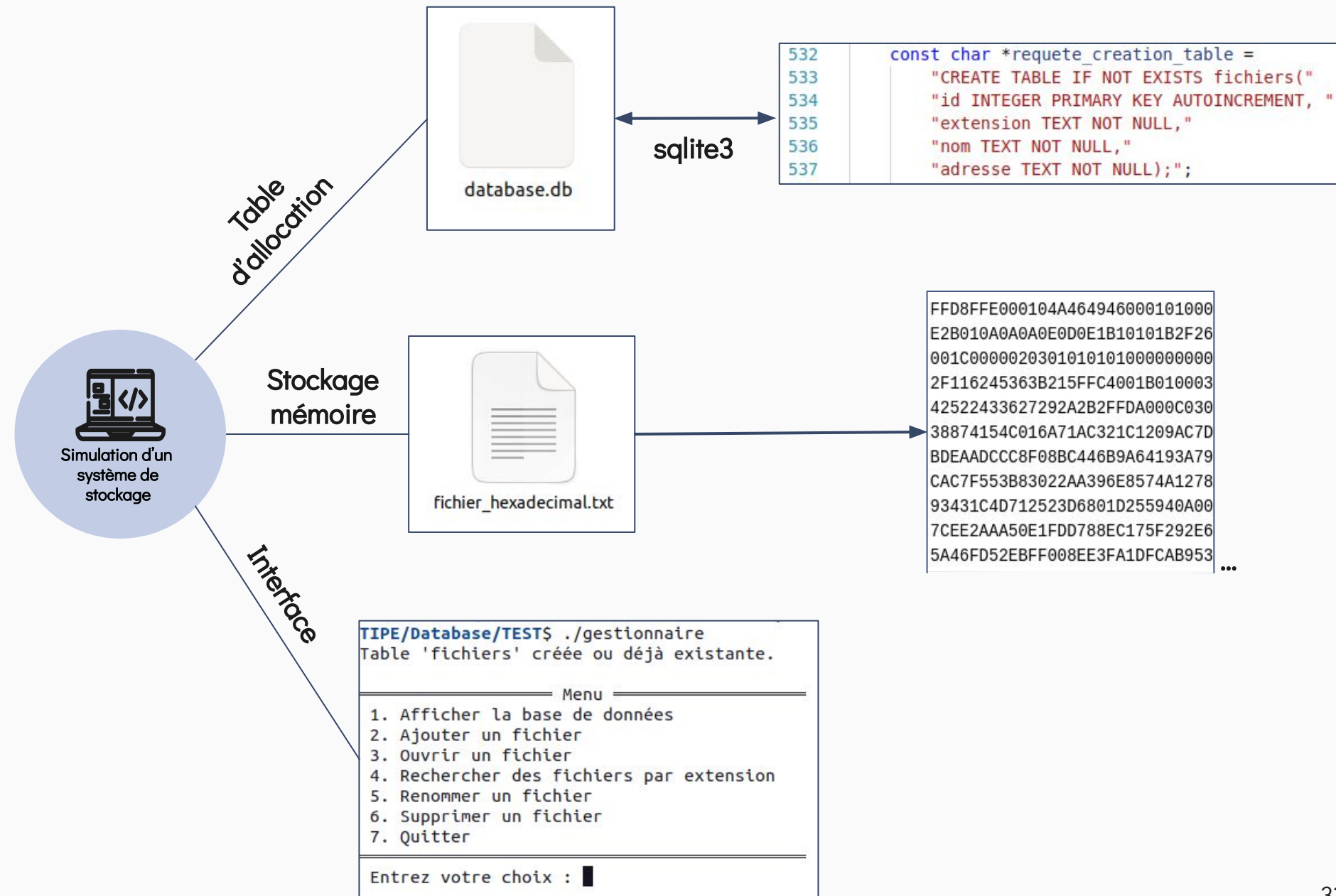
2.1 Création et utilisation de la base



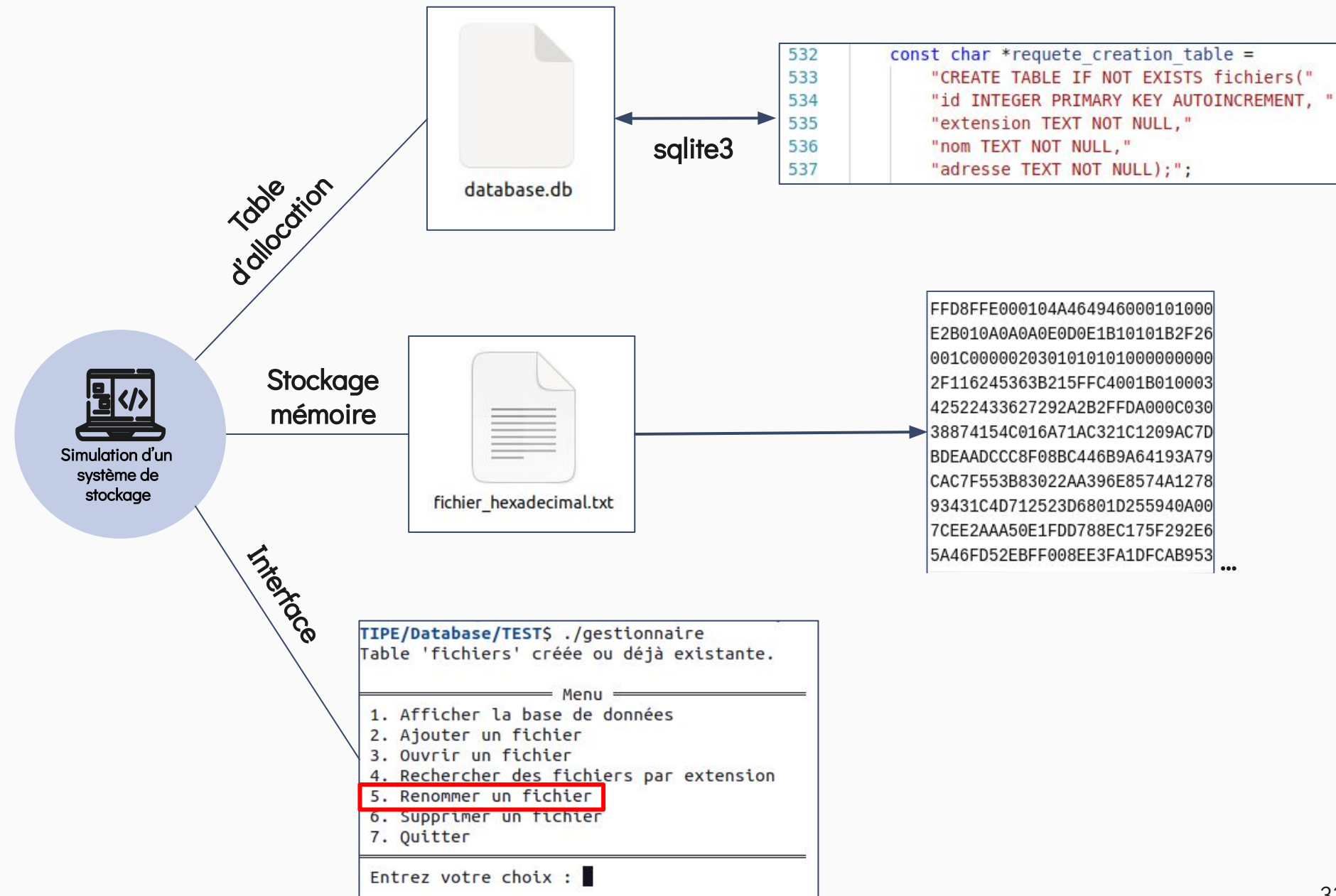
2: 2 Représentation stockage mémoire



2: Simulation d'un système de stockage



2: Simulation d'un système de stockage



Injection SQL:

```
/TEST$ ./gestionnaire  
Table 'fichiers' créée ou déjà existante.
```

Menu

1. Afficher la base de données
2. Ajouter un fichier
3. Ouvrir un fichier
4. Rechercher des fichiers par extension
5. Renommer un fichier
6. Supprimer un fichier
7. Quitter

Entrez votre choix : 5

VULNÉRABLE !

===== Renommer =====

ID du fichier à renommer : 1

Nouveau nom du fichier : "Hacked"; DROP TABLE fichiers; --"

id	extension	nom
Erreur lors de la sélection des données : no such table: fichiers		

Menu

1. Afficher la base de données
2. Ajouter un fichier
3. Ouvrir un fichier
4. Rechercher des fichiers par extension
5. Renommer un fichier
6. Supprimer un fichier
7. Quitter

Entrez votre choix :

La donnée est injectée dans le texte SQL, donc **interprétée comme du code**.

Injection SQL:

```
405 void renommer(sqlite3 *db, ftype *fichier) {
406     sqlite3_stmt *stmt;
407     char requete[256];
408
409     // On garde le %d car un int est sûr à injecter dans une chaîne SQL
410     snprintf(requete, sizeof(requete), "UPDATE fichiers SET nom = ? WHERE id = %d;", fichier->id);
411
412     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) != SQLITE_OK) {
413         fprintf(stderr, "Erreur préparation requête : %s\n", sqlite3_errmsg(db));
414         return;
415     }
416
417     // Sécurise uniquement le paramètre texte (le nom), pas besoin de bind sur l'ID ici
418     sqlite3_bind_text(stmt, 1, fichier->nom, -1, SQLITE_TRANSIENT);
419
420     if (sqlite3_step(stmt) != SQLITE_DONE) {
421         fprintf(stderr, "Erreur exécution requête : %s\n", sqlite3_errmsg(db));
422     }
423
424     sqlite3_finalize(stmt);
425 }
```

Injection SQL:

Pourquoi on est maintenant protégé ?

```
sqlite3_prepare_v2(db, "UPDATE fichiers SET nom = ?;", -1, &stmt, NULL);  
sqlite3_bind_text(stmt, 1, nom, -1, SQLITE_TRANSIENT);
```

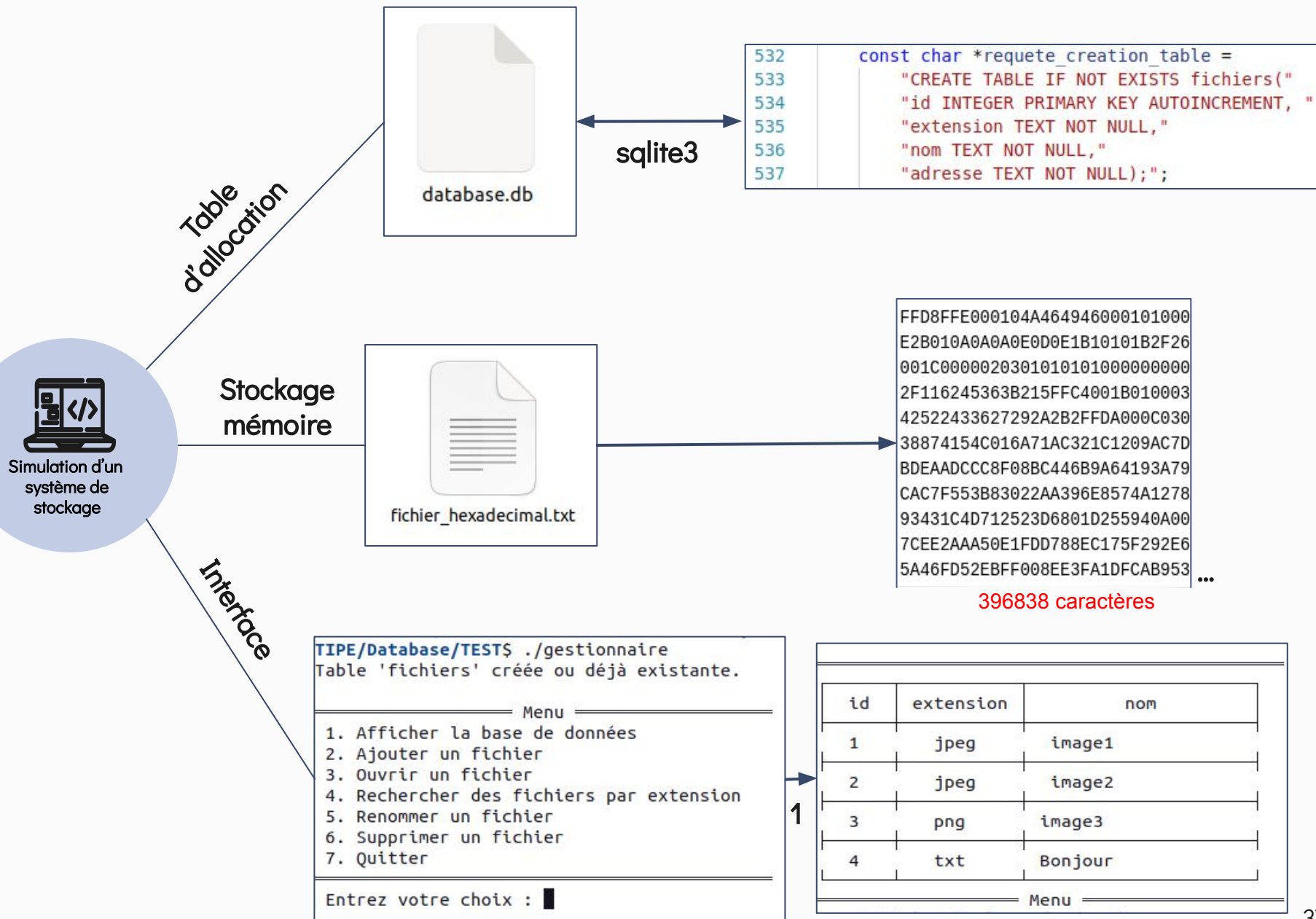
Ex avec `'"Hacked'; DROP TABLE fichiers; --"`

```
"UPDATE fichiers SET nom = "Hacked"; DROP TABLE fichiers; --";"
```

On exécute la commande comme
précédemment

Ici, la chaîne `'Hacked'; DROP TABLE fichiers; --` est insérée **comme une donnée dans le champ nom**, et jamais exécutée.

2: Simulation d'un système de stockage



Exemple:

Nom	Taille
 database.db	12,3 ko
 fichier_hexadecimal.txt	396,8 ko
 gestionnaire	34,8 ko
 gestionnaire-fichier-simulation.c	21,9 ko
 reconstitution	16,8 ko
 reconstitution_fichier.c	6,6 ko

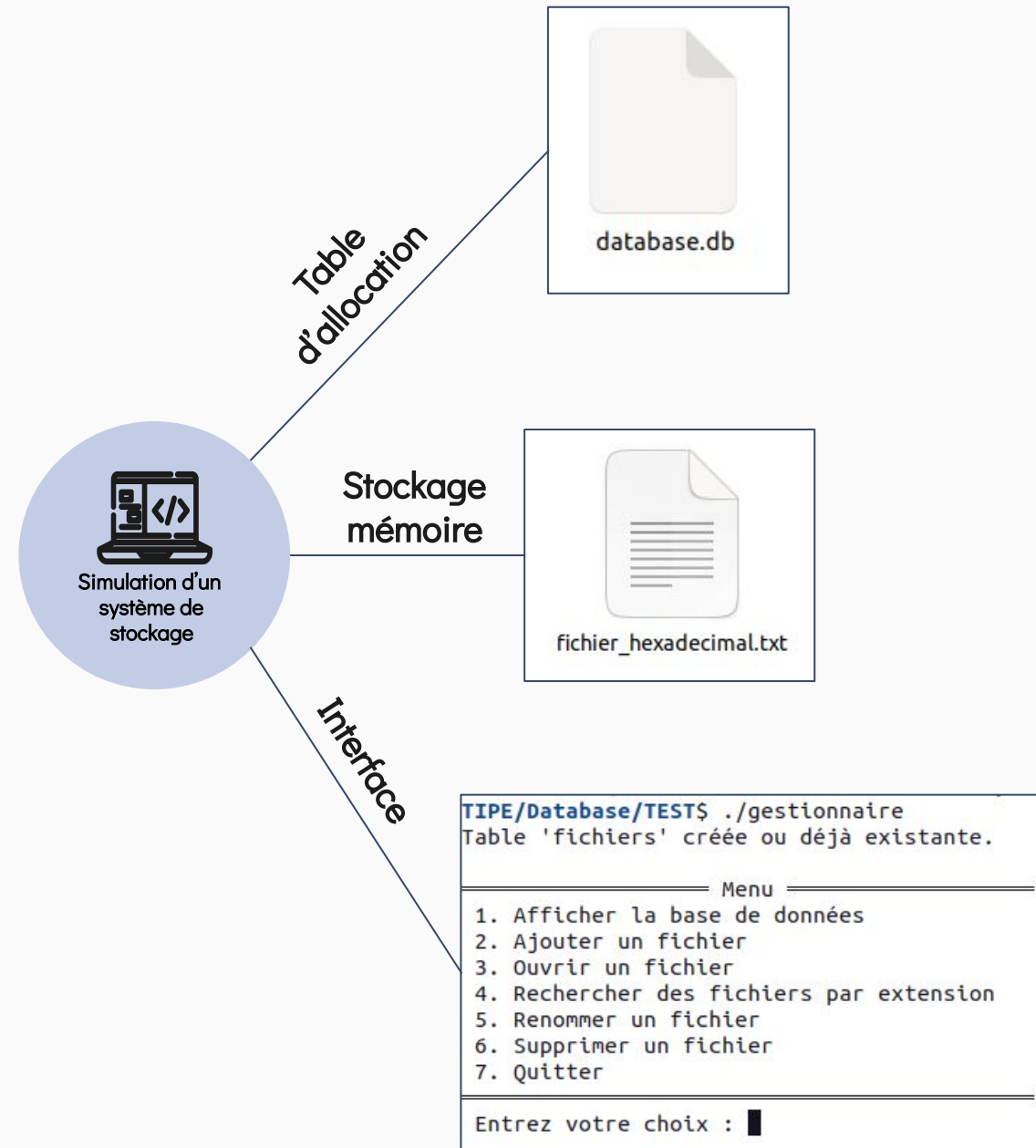
Exemple:

Nom		Taille
	fichier_hexadecimal.txt	396,8 ko
	reconstitution	16,8 ko
	reconstitution_fichier.c	6,6 ko

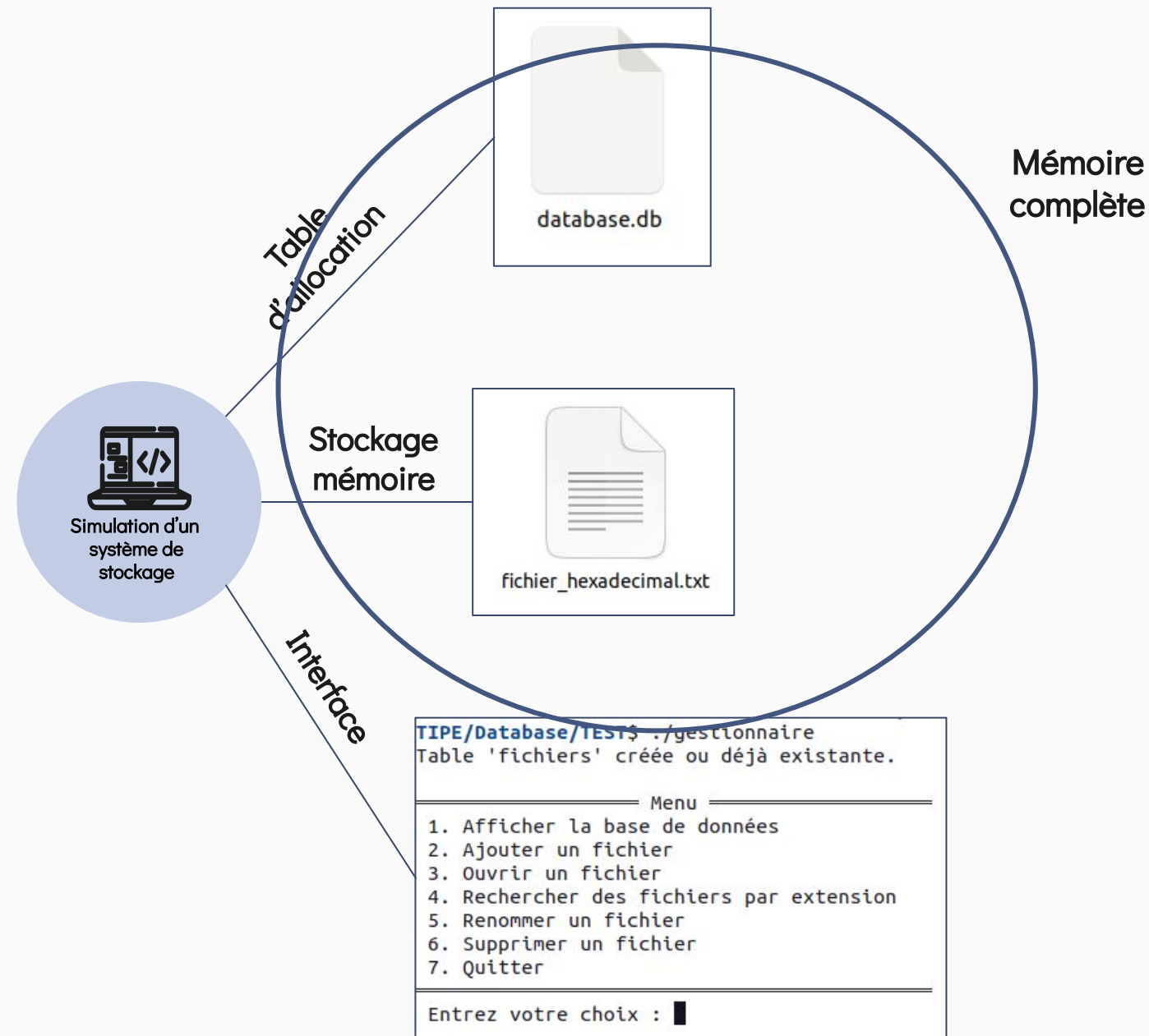
Exemple:

Nom	Taille
 fichier1.jpeg	13,1 ko
 fichier1.png	177,4 ko
 fichier2.jpeg	7,9 ko
 fichier3.jpeg	20,8 ko
 fichier_hexadecimal.txt	396,8 ko
 reconstitution	16,8 ko
 reconstitution_fichier.c	6,6 ko

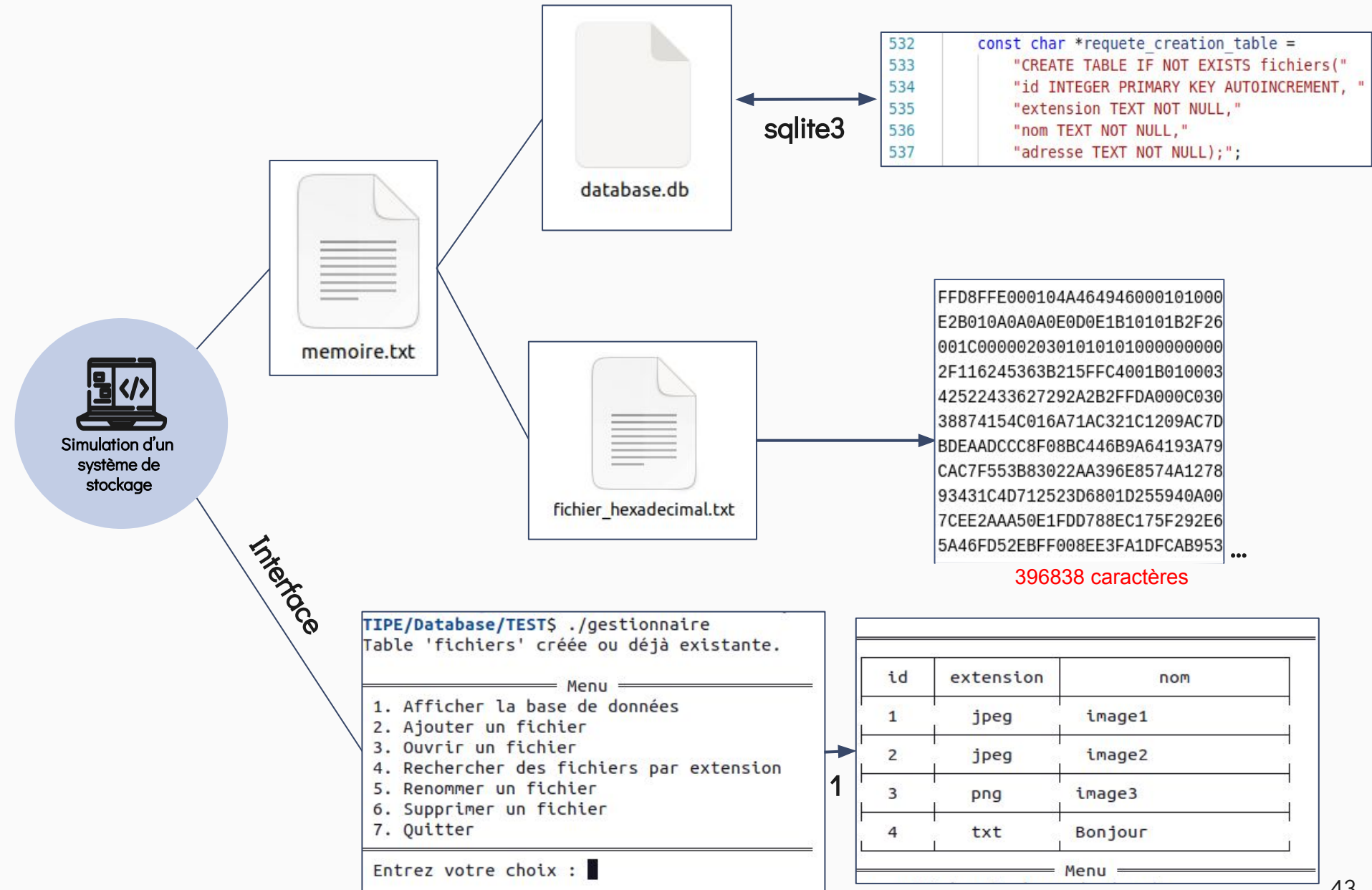
2: Simulation d'un système de stockage



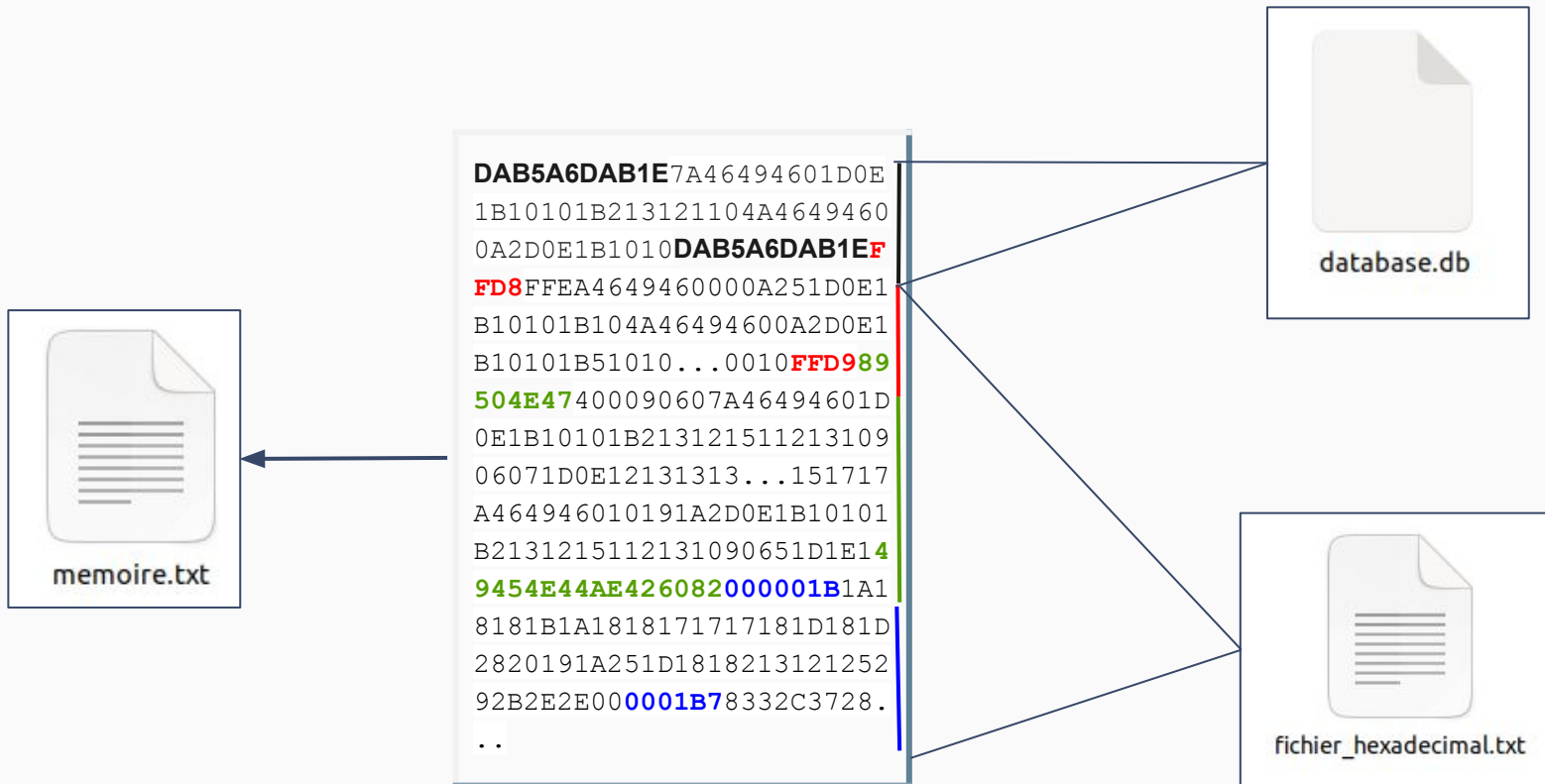
2: Simulation d'un système de stockage



3: Récupérer l'arborescence de fichiers



3: Récupérer l'arborescence de fichiers



Pour faire un bilan:

Pour faire un bilan:

Actions accomplies:

- Récupération de données **en théorie possible**. (JPEG, PNG, MOV, GIF, etc...)
- **Simulation** et **algorithmes** pour traiter les données bruts et de récupérer les métadonnées.

Pour faire un bilan:

Actions accomplies:

- Récupération de données **en théorie possible**. (JPEG, PNG, MOV, GIF, etc...)
- **Simulation** et **algorithmes** pour traiter les données bruts et de récupérer les métadonnées.

Prochaines étapes :

- Simulation avec **partition** de la mémoire et **vrai arborescence** (dossiers)
- **Métadonnées plus importantes** pour être un **vrai système de stockage** (date de création, dernière modification, créateur du fichier, ...)

Merci de m'avoir écouté !

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)

Please keep this slide for attribution

Sources MCOT

id	titre	auteur	lien web
1	Understanding Deleted Files and What They Mean	Steven Alexander	https://www.hgexperts.com/expert-witness-articles/understanding-deleted-files-and-what-they-mean-44950
2	Scalpel: A Frugal, High Performance File Carver	Golden Richard III	https://dfrws.org/presentation/scalpel-a-frugal-high-performance-file-carver/
3	GCK'S FILE SIGNATURES TABLE	Gary Kessler	https://www.garykessler.net/library/file_signatures.html
4	All About GIF89a	Royal E. Frazier Jr.	https://web.archive.org/web/19990418091037/http://www6.uniovi.es/gifanim/gifabout.htm
5	Comment vérifier l'intégrité d'un fichier ?	Kevin ENGEL	https://www.tech2tech.fr/comment-verifier-lintegrite-dun-fichier/
6	Performance Analysis of File Carving Tools	Thomas Laurenson	https://inria.hal.science/hal-01463843/document
7	Modern Operating Systems	ANDREW S. TANENBAUM HERBERT BOS	http://stst.elia.pub.ro/news/SO/Modern%20Operating%20System%20-%20Tanenbaum.pdf
8	SQL Injection Attacks: Techniques and Protection Mechanisms	Nikita Patel et al.	https://www.academia.edu/download/101022407/IJCSE11-03-01-139.pdf

Fonctions de <string.h> :

```
int snprintf(char *str, size_t size, const char *format, ...);
```

- Écrit une chaîne formatée dans un tampon `char\*`. Le nombre maximal de caractères écrits est limité par la taille spécifiée dans `size`.

```
char *strncpy(char *dest, const char *src, size_t n);
```

- Copie une chaîne de caractères d'une source vers une destination, avec une limite sur le nombre de caractères copiés. Si la source est plus courte que `n`, le reste de la destination sera rempli avec des caractères nuls (`\0`).

```
char *strrchr(const char *str, int c);
```

- Recherche la dernière occurrence d'un caractère dans une chaîne de caractères.

```
void *memcpy(void *dest, const void *src, size_t n);
```

- Copie un bloc de mémoire brute d'un emplacement à un autre. Attention, `**`memcpy`**` ne vérifie pas les zones mémoire chevauchantes, ce qui peut causer des comportements indéfinis.

Fonctions de <stdio.h> :

```
size_t fread(void *ptr, size_t size, size_t count, FILE *stream);
```

- Lit des données depuis un fichier vers un buffer en mémoire. Retourne le nombre d'éléments effectivement lus. Si ce nombre est inférieur à `count`, cela peut être dû à une erreur ou à la fin du fichier.

```
void rewind(FILE *stream);
```

- Repositionne le pointeur de lecture/écriture d'un fichier au début et efface l'indicateur d'erreur.

```
int fseek(FILE *stream, long offset, int origin);
```

- Déplace le pointeur de lecture/écriture dans un fichier à une position spécifiée par `offset` par rapport à `origin` (`SEEK\_SET`, `SEEK\_CUR`, `SEEK\_END`).

```
long ftell(FILE *stream);
```

- Retourne la position actuelle du pointeur de lecture/écriture dans le fichier, exprimée en octets depuis le début

```
int fputc(int c, FILE *stream);
```

- Écrit un caractère dans un fichier. Le caractère est passé sous forme d'un entier, mais il est converti en `char` lors de l'écriture

```
int getchar(void);
```

- Lit un caractère depuis l'entrée standard, souvent pour consommer le retour à la ligne après un scanf

Fonctions SQLite3 :

sqlite3 *db: un pointeur vers la base de données SQLite

```
int sqlite3_open(const char *filename, sqlite3 **db);
```

- Ouvre une base de données SQLite. Si l'ouverture échoue, un message d'erreur peut être récupéré avec `**`sqlite3_errmsg`**`.

```
int sqlite3_close(sqlite3 *db);
```

- Ferme une base de données SQLite. Retourne ``SQLITE_OK`` si la fermeture est réussie, sinon retourne un code d'erreur.

```
int sqlite3_prepare_v2(sqlite3 *db, const char *sql, int nByte, sqlite3_stmt **stmt, const char **tail);
```

- Prépare une requête SQL et compile la chaîne SQL en une instruction SQLite prête à être exécutée. Retourne ``SQLITE_OK`` si la préparation est réussie.

```
int sqlite3_step(sqlite3_stmt *stmt);
```

- Exécute une requête SQL préparée. Cette fonction retourne plusieurs codes :

- ``SQLITE_ROW`` : une ligne de résultat est disponible.
- ``SQLITE_DONE`` : la requête est terminée (par exemple, une requête ``INSERT`` ou ``UPDATE``).

```
int sqlite3_finalize(sqlite3_stmt *stmt);
```

- Libère les ressources utilisées par une instruction SQL préparée (``sqlite3_stmt``). Retourne ``SQLITE_OK`` si la finalisation est réussie.

```
const char *sqlite3_errmsg(sqlite3 *db);
```

- Récupère un message d'erreur détaillé associé à la dernière erreur SQLite sur la base de données.

Fonctions SQLite3 :

```
void sqlite3_free(void *ptr);
```

- Libère la mémoire allouée par SQLite pour certains objets, comme les résultats de requêtes.

```
int sqlite3_exec(sqlite3 *db, const char *sql, int (*callback)(void*,int,char**,char**), void *arg, char **errmsg);
```

- Exécute une requête SQL simple (par exemple un `SELECT`). Si une fonction de rappel (`callback`) est fournie, elle est appelée pour chaque ligne de résultat. Retourne `SQLITE\_OK` si l'exécution est réussie.

```
const unsigned char *sqlite3_column_text(sqlite3_stmt *stmt, int col);
```

- Récupère les données d'une colonne de type texte à partir du résultat d'une requête SQL. Retourne un pointeur vers la chaîne de caractères.

```
int sqlite3_column_int64(sqlite3_stmt *stmt, int col);
```

- Récupère les données d'une colonne de type entier sur 64 bits dans une requête SQL.

```
int sqlite3_bind_text(sqlite3_stmt *stmt, int index, const char *value, int length, void (*destructor)(void*));
```

- Lie une valeur texte à un paramètre (`?`) dans une requête SQL préparée.
`index` commence à 1. `value` est la chaîne à insérer, `length` sa taille (ou -1 pour longueur automatique).
`destructor` indique si SQLite doit copier ou libérer la mémoire après exécution (ex: `SQLITE\_TRANSIENT` pour copier).

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sqlite3.h>
4  #include <string.h>
5
6  /* TIPE 2024-2025, MPI Lycée Louis Thuillier: Cardot Clément et William Guerrin-Garnier
7
8  Objectifs: simuler un gestionnaire de fichiers
9      - accès à une base de données
10     - opération sur la base classique
11     - accès brut à la base de données
12
13 ATTENTION: Pour compiler, il faut inclure sqlite en compilant:
14     "gcc gestionnaire-fichier-simulation.c -o [Nom-gestionnaire] -lsqlite3"
15
16 */
17
18 //===== FONCTION POUR 'ftype' =====
19
20 typedef struct {
21     int id;
22     char nom[128];
23     char extension[16];
24     char adresse[256];
25     long offset;
26     long longueur;
27 } ftype;
28
29 void nom_data(sqlite3 *db, ftype *fichier) {
30     char requete[512];
31     snprintf(requete, sizeof(requete), "SELECT nom FROM fichiers WHERE id = %d;", fichier->id);
32
33     sqlite3_stmt *stmt;
34     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {
35         if (sqlite3_step(stmt) == SQLITE_ROW) {
36             const char *nom = (const char *)sqlite3_column_text(stmt, 0);
37             strncpy(fichier->nom, nom, sizeof(fichier->nom));
38         }
39     }
40     sqlite3_finalize(stmt);
41 }
42
43 void extension_data(sqlite3 *db, ftype *fichier) {
44     char requete[512];
45     snprintf(requete, sizeof(requete), "SELECT extension FROM fichiers WHERE id = %d;", fichier->id);
46
47     sqlite3_stmt *stmt;
48     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {
49         if (sqlite3_step(stmt) == SQLITE_ROW) {
50             const char *extension = (const char *)sqlite3_column_text(stmt, 0);
51             strncpy(fichier->extension, extension, sizeof(fichier->extension));
52         }
53     }
54     sqlite3_finalize(stmt);
55 }
56

```

```
57 void adresse_data(sqlite3 *db, ftype *fichier) {
58     char requete[512];
59     snprintf(requete, sizeof(requete), "SELECT adresse FROM fichiers WHERE id = %d;", fichier->id);
60
61     sqlite3_stmt *stmt;
62     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {
63         if (sqlite3_step(stmt) == SQLITE_ROW) {
64             const char *adresse = (const char *)sqlite3_column_text(stmt, 0);
65             strncpy(fichier->adresse, adresse, sizeof(fichier->adresse));
66         }
67     }
68     sqlite3_finalize(stmt);
69 }
70
71 void offset_data(sqlite3 *db, ftype *fichier) {
72     char requete[512];
73     snprintf(requete, sizeof(requete), "SELECT offset FROM fichiers WHERE id = %d;", fichier->id);
74
75     sqlite3_stmt *stmt;
76     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {
77         if (sqlite3_step(stmt) == SQLITE_ROW) {
78             fichier->offset = sqlite3_column_int64(stmt, 0);
79         }
80     }
81     sqlite3_finalize(stmt);
82 }
83
84 void longueur_data(sqlite3 *db, ftype *fichier) {
85     char requete[512];
86     snprintf(requete, sizeof(requete), "SELECT longueur FROM fichiers WHERE id = %d;", fichier->id);
87
88     sqlite3_stmt *stmt;
89     if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) == SQLITE_OK) {
90         if (sqlite3_step(stmt) == SQLITE_ROW) {
91             fichier->longueur = sqlite3_column_int64(stmt, 0);
92         }
93     }
94     sqlite3_finalize(stmt);
95 }
96
```

```
97 //===== FONCTION SQL / HEXA =====
98
99 void extraire_nom_et_extension(ftype *fichier) {
100     const char *nom_fichier = strrchr(fichier->adresse, '/'); // Trouver le dernier '/'
101     if (!nom_fichier) {
102         nom_fichier = fichier->adresse; // Si aucun '/' trouvé, utiliser tout le adresse
103     } else {
104         nom_fichier++; // Passer le '/'
105     }
106
107     const char *ext = strrchr(nom_fichier, '.');
108     if (ext) {
109         strcpy(fichier->extension, ext + 1); // Copier l'extension sans le point
110         strncpy(fichier->nom, nom_fichier, ext - nom_fichier); // Copier le nom sans l'extension
111         fichier->nom[ext - nom_fichier] = '\0'; // Terminer la chaîne de caractères
112     } else {
113         strcpy(fichier->nom, nom_fichier); // Pas d'extension trouvée
114         strcpy(fichier->extension, "");
115     }
116 }
117
118 int convertir_fichier_en_hexa(ftype *fichier) {
119     FILE *fichier_bin = fopen(fichier->adresse, "rb");
120     if (!fichier_bin) {
121         fprintf(stderr, "Erreur d'ouverture du fichier : %s\n", fichier->adresse);
122         return -1;
123     }
124     FILE *fichier_hexa = fopen("data_hexa.txt", "a");
125     if (!fichier_hexa) {
126         fprintf(stderr, "Erreur d'ouverture du fichier data_hexa.txt\n");
127         fclose(fichier_bin);
128         return -1;
129     }
130     unsigned char octet;
131     while (fread(&octet, 1, 1, fichier_bin) == 1) {
132         fprintf(fichier_hexa, "%02X", octet);
133     }
134     fclose(fichier_bin);
135     fclose(fichier_hexa);
136     return 0;
137 }
138
139 int supprimer_fichier(const char *adresse) {
140     if (remove(adresse) == 0) {
141         printf(" Fichier '%s' supprimé avec succès.\n", adresse);
142         return 0;
143     } else {
144         perror(" Erreur de suppression du fichier");
145         return -1;
146     }
147 }
```

```

149 int ajouter_fichier(sqlite3 *database, ftype *fichier) {
150     char *message_erreur = 0;
151     char requete[512];
152     long offset;
153     long longueur = 0;
154
155     // Extraire le nom et l'extension
156     extraire_nom_et_extension(fichier);
157
158     // Ouvrir le fichier en hexadécimal et déterminer l'offset
159     FILE *fichier_hexa = fopen("fichier_hexadecimal.txt", "a");
160     if (!fichier_hexa) {
161         fprintf(stderr, "Erreur d'ouverture du fichier fichier_hexadecimal.txt\n");
162         return -1;
163     }
164     offset = ftell(fichier_hexa); // Obtenir l'offset actuel
165
166     FILE *fichier_bin = fopen(fichier->adresse, "rb");
167     if (!fichier_bin) {
168         fprintf(stderr, "Erreur d'ouverture du fichier : %s\n", fichier->adresse);
169         fclose(fichier_hexa);
170         return -1;
171     }
172
173     unsigned char octet;
174     while (fread(&octet, 1, 1, fichier_bin) == 1) {
175         fprintf(fichier_hexa, "%02X", octet);
176         longueur++;
177     }
178     fclose(fichier_bin);
179     fclose(fichier_hexa);
180
181     // Mettre à jour l'offset et la longueur dans la structure
182     fichier->offset = offset;
183     fichier->longueur = longueur;
184
185     // Insérer dans la base de données avec l'offset et la longueur
186     snprintf(requete, sizeof(requete),
187             "INSERT INTO fichiers (nom, extension, adresse) VALUES ('%s', '%s', '%ld:%ld');",
188             fichier->nom, fichier->extension, fichier->offset, fichier->longueur);
189
190     int resultat = sqlite3_exec(database, requete, 0, 0, &message_erreur);
191     if (resultat != SQLITE_OK) {
192         fprintf(stderr, "Erreur d'insertion : %s\n", message_erreur);
193         sqlite3_free(message_erreur);
194         return resultat;
195     }
196
197     fprintf(stdout, "Fichier '%s' avec extension '%s' ajouté avec succès.\n\n", fichier->nom, fichier->extension);
198
199     if (supprime_fichier(fichier->adresse) != 0) {
200         fprintf(stderr, "Erreur lors de la suppression du fichier : %s\n", fichier->adresse);
201     }
202     return SQLITE_OK;
203 }
204

```

```
205 int ouvrir_data(sqlite3 *database, ftype *fichier) {
206     char requete[256];
207     char *message_erreur = 0;
208     sqlite3_stmt *stmt;
209
210     snprintf(requete, sizeof(requete), "SELECT adresse FROM fichiers WHERE id = '%d';", fichier->id);
211
212     if (sqlite3_prepare_v2(database, requete, -1, &stmt, 0) != SQLITE_OK) {
213         fprintf(stderr, "Erreur de préparation de la requête : %s\n", sqlite3_errmsg(database));
214         return -1;
215     }
216
217     if (sqlite3_step(stmt) == SQLITE_ROW) {
218         const char *adresse = (const char *)sqlite3_column_text(stmt, 0);
219         sscanf(adresse, "%ld:%ld", &(fichier->offset), &(fichier->longueur));
220     } else {
221         fprintf(stderr, "Fichier non trouvé dans la base de données.\n");
222         sqlite3_finalize(stmt);
223         return -1;
224     }
225     sqlite3_finalize(stmt);
226
227     FILE *fichier_hexa = fopen("fichier_hexadecimal.txt", "r");
228     if (!fichier_hexa) {
229         fprintf(stderr, "Erreur d'ouverture du fichier fichier_hexadecimal.txt\n");
230         return -1;
231     }
232
233     char nom_complet[256];
234     sprintf(nom_complet, "%s.%s", fichier->nom, fichier->extension);
235     FILE *fichier_sortie = fopen(nom_complet, "wb");
236     if (!fichier_sortie) {
237         fprintf(stderr, "Erreur de création du fichier : %s.%s\n", fichier->nom, fichier->extension);
238         fclose(fichier_hexa);
239         return -1;
240     }
241
242     fseek(fichier_hexa, fichier->offset, SEEK_SET);
243     for (long i = 0; i < fichier->longueur; i++) {
244         unsigned int octet;
245         fscanf(fichier_hexa, "%02X", &octet);
246         fputc(octet, fichier_sortie);
247     }
248
249     fclose(fichier_sortie);
250     fclose(fichier_hexa);
251     return 0;
252 }
253
```

```
254 int supprimer(sqlite3 *database, ftype *fichier) {
255     char requete[256];
256     char *message_erreur = 0;
257     sqlite3_stmt *stmt;
258     long offset, longueur;
259
260     // 1. Récupérer l'offset et la longueur du fichier à supprimer
261     snprintf(requete, sizeof(requete), "SELECT adresse FROM fichiers WHERE id = '%d';", fichier->id);
262     if (sqlite3_prepare_v2(database, requete, -1, &stmt, 0) != SQLITE_OK) {
263         fprintf(stderr, "Erreur de préparation de la requête : %s\n", sqlite3_errmsg(database));
264         return -1;
265     }
266
267     if (sqlite3_step(stmt) == SQLITE_ROW) {
268         const char *adresse = (const char *)sqlite3_column_text(stmt, 0);
269         sscanf(adresse, "%ld:%ld", &offset, &longueur);
270     } else {
271         fprintf(stderr, "Fichier non trouvé dans la base de données.\n");
272         sqlite3_finalize(stmt);
273         return -1;
274     }
275     sqlite3_finalize(stmt);
276
277     // 2. Supprimer l'entrée de la base de données
278     snprintf(requete, sizeof(requete), "DELETE FROM fichiers WHERE id = '%d';", fichier->id);
279     int resultat = sqlite3_exec(database, requete, 0, 0, &message_erreur);
280     if (resultat != SQLITE_OK) {
281         fprintf(stderr, "Erreur de suppression : %s\n", message_erreur);
282         sqlite3_free(message_erreur);
283         return resultat;
284     }
285
286     // 3. Lire le contenu du fichier hexadecimal dans un buffer (nécessaire pour le réécrire ensuite)
287     FILE *fichier_hexa = fopen("fichier_hexadecimal.txt", "r");
288     if (!fichier_hexa) {
289         fprintf(stderr, "Erreur d'ouverture du fichier fichier_hexadecimal.txt\n");
290         return -1;
291     }
292
293     fseek(fichier_hexa, 0, SEEK_END);
294     long taille_totale = ftell(fichier_hexa);
295     rewind(fichier_hexa);
296
297     char *buffer = malloc(taille_totale + 1);
298     if (!buffer) {
299         fprintf(stderr, "Erreur d'allocation mémoire.\n");
300         fclose(fichier_hexa);
301         return -1;
302     }
303     fread(buffer, 1, taille_totale, fichier_hexa);
304     fclose(fichier_hexa);
305
306     // 4. Supprimer les données du fichier à supprimer en compactant le buffer
307     long nouvelle_taille = taille_totale - (longueur * 2);
308     char *nouveau_buffer = malloc(nouvelle_taille + 1);
309     if (!nouveau_buffer) {
310         fprintf(stderr, "Erreur d'allocation mémoire.\n");
311         free(buffer);
312         return -1;
313     }
```

```

254 int supprimer(sqlite3 *database, ftype *fichier) {
309     if (!nouveau_buffer) {
313     }
314
315     memcpy(nouveau_buffer, buffer, offset);
316     memcpy(nouveau_buffer + offset, buffer + offset + (longueur * 2), taille_totale - offset - (longueur * 2));
317
318     free(buffer);
319
320     // 5. Réécrire le fichier hexadecimal avec le nouveau contenu
321     fichier_hexa = fopen("fichier_hexadecimal.txt", "w");
322     if (!fichier_hexa) {
323         fprintf(stderr, "Erreur d'ouverture en écriture du fichier fichier_hexadecimal.txt\n");
324         free(nouveau_buffer);
325         return -1;
326     }
327
328     fwrite(nouveau_buffer, 1, nouvelle_taille, fichier_hexa);
329     fclose(fichier_hexa);
330     free(nouveau_buffer);
331
332     // 6. Mettre à jour les offsets des autres fichiers dans la base de données
333     snprintf(requete, sizeof(requete), "SELECT id, adresse FROM fichiers;");
334     if (sqlite3_prepare_v2(database, requete, -1, &stmt, 0) != SQLITE_OK) {
335         fprintf(stderr, "Erreur de préparation de la requête : %s\n", sqlite3_errmsg(database));
336         return -1;
337     }
338
339     while (sqlite3_step(stmt) == SQLITE_ROW) {
340         int id = sqlite3_column_int(stmt, 0);
341         const char *adresse = (const char *)sqlite3_column_text(stmt, 1);
342         long fichier_offset, fichier_longueur;
343         sscanf(adresse, "%ld:%ld", &fichier_offset, &fichier_longueur);
344
345         // Si l'offset du fichier se situe après celui qu'on vient de supprimer, on le décale
346         if (fichier_offset > offset) {
347             fichier_offset -= (longueur * 2); // Mettre à jour l'offset
348
349             // Mettre à jour l'entrée dans la base de données
350             snprintf(requete, sizeof(requete),
351                 "UPDATE fichiers SET adresse = '%ld:%ld' WHERE id = %d;",
352                 fichier_offset, fichier_longueur, id);
353
354             resultat = sqlite3_exec(database, requete, 0, 0, &message_erreur);
355             if (resultat != SQLITE_OK) {
356                 fprintf(stderr, "Erreur de mise à jour : %s\n", message_erreur);
357                 sqlite3_free(message_erreur);
358                 sqlite3_finalize(stmt);
359                 return resultat;
360             }
361         }
362     }
363     sqlite3_finalize(stmt);
364
365     printf("Fichier '%s' supprimé avec succès.\n", fichier->nom);
366     return 0;
367 }
368
369

```



```
427  /*
428  void renommer(sqlite3 *database, ftype *fichier) {
429      int resultat;
430      char *message_erreur = 0;
431      char requete_selection[256];
432      snprintf(requete_selection, sizeof(requete_selection), "UPDATE fichiers SET nom = '%s' WHERE id = %d;", fichier->nom, fic
433      //snprintf: printf dans un char* tampon (permet de recevoir le "%s")
434      resultat = sqlite3_exec(database, requete_selection, fonction_de_rappel, 0, &message_erreur);
435      if (resultat != SQLITE_OK) {
436          fprintf(stderr, "Erreur lors de la sélection des données : %s\n", message_erreur);
437          sqlite3_free(message_erreur);
438      }
439      snprintf(requete_selection, sizeof(requete_selection), "SELECT * FROM fichiers WHERE id = %d;", fichier->id);
440      affiche_tab(database, requete_selection);
441  }
442
443
444  // ⚠ VULNÉRABLE à l'injection SQL
445  void renommer_insecure(sqlite3 *database, ftype *fichier) {
446      int resultat;
447      char *message_erreur = 0;
448      char requete_selection[512];
449
450      // concaténation directe de chaîne non échappée
451      sprintf(requete_selection, "UPDATE fichiers SET nom = '%s' WHERE id = %d;", fichier->nom, fichier->id);
452
453      resultat = sqlite3_exec(database, requete_selection, NULL, 0, &message_erreur);
454      if (resultat != SQLITE_OK) {
455          fprintf(stderr, "Erreur SQL : %s\n", message_erreur);
456          sqlite3_free(message_erreur);
457      }
458  }
459  // ✅ SÉCURISÉE avec requête préparée
460  void renommer(sqlite3 *db, ftype *fichier) {
461      sqlite3_stmt *stmt;
462      char requete[256];
463
464      // On garde le %d car un int est sûr à injecter dans une chaîne SQL
465      snprintf(requete, sizeof(requete), "UPDATE fichiers SET nom = ? WHERE id = %d;", fichier->id);
466
467      if (sqlite3_prepare_v2(db, requete, -1, &stmt, NULL) != SQLITE_OK) {
468          fprintf(stderr, "Erreur préparation requête : %s\n", sqlite3_errmsg(db));
469          return;
470      }
471
472      // Sécurise uniquement le paramètre texte (le nom), pas besoin de bind sur l'ID ici
473      sqlite3_bind_text(stmt, 1, fichier->nom, -1, SQLITE_TRANSIENT);
474
475      if (sqlite3_step(stmt) != SQLITE_DONE) {
476          fprintf(stderr, "Erreur exécution requête : %s\n", sqlite3_errmsg(db));
477      }
478
479      sqlite3_finalize(stmt);
480  }
481
482
483  */
```

```

485 void interface(sqlite3 *database) {
486     int choix;
487     ftype *fichier = malloc(sizeof(ftype));
488
489     while (1) {
490         printf("\n===== Menu =====\n");
491         printf(" 1. Afficher la base de données\n");
492         printf(" 2. Ajouter un fichier\n");
493         printf(" 3. Ouvrir un fichier\n");
494         printf(" 4. Rechercher des fichiers par extension\n");
495         printf(" 5. Renommer un fichier\n");
496         printf(" 6. Supprimer un fichier\n");
497         printf(" 7. Quitter\n");
498         printf("===== \n");
499
500         printf(" Entrez votre choix : ");
501         scanf("%d", &choix);
502         getchar(); // Pour consommer le retour à la ligne laissé par scanf
503
504         switch (choix) {
505             case 1:
506                 ls_database(database);
507                 break;
508
509             case 2:
510                 printf("\n\n===== Ajout =====\n");
511                 printf(" adresse du fichier à ajouter : ");
512                 fgets(fichier->adresse, sizeof(fichier->adresse), stdin);
513                 fichier->adresse[strcspn(fichier->adresse, "\n")] = 0; // Enlever le retour à la ligne
514                 if (ajouter_fichier(database, fichier) == 0) {
515                     printf("\n\n Fichier ajouté avec succès.\n\n");
516                 } else {
517                     printf("\n\n Erreur lors de l'ajout du fichier.\n\n");
518                 }
519                 break;
520
521             case 3:
522                 printf("\n\n===== Ouvrir =====\n");
523                 printf(" ID du fichier à ouvrir : ");
524                 scanf("%d", &(fichier->id));
525                 getchar(); // Pour consommer le retour à la ligne laissé par scanf
526                 // Remplir le nom, l'extension, l'adresse, et l'offset
527                 nom_data(database, fichier);
528                 extension_data(database, fichier);
529                 adresse_data(database, fichier);
530                 offset_data(database, fichier);
531
532                 // Après récupération des données, essayez d'ouvrir le fichier
533                 if (ouvrir_data(database, fichier) == 0) {
534                     printf("\n\n Fichier ouvert et créé avec succès.\n\n");
535                 } else {
536                     printf("\n\n Erreur lors de l'ouverture du fichier.\n\n");
537                 }
538                 break;
539
540             default:
541                 break;
542         }
543     }
544 }

```

```
485 void interface(sqlite3 *database) {
486     while (1) {
487         switch (choix) {
488             case 3:
489                 break;
490
491             case 4:
492                 printf("\n\n===== Recherche =====\n");
493                 printf(" Extension des fichiers à rechercher : ");
494                 fgets(fichier->extension, sizeof(fichier->extension), stdin);
495                 fichier->extension[strcspn(fichier->extension, "\n")] = 0;
496                 recherche(database, fichier);
497                 break;
498
499             case 5:
500                 printf("\n\n===== Renommer =====\n");
501                 printf(" ID du fichier à renommer : ");
502                 scanf("%d", &(fichier->id));
503                 getchar(); // Pour consommer le retour à la ligne laissé par scanf
504                 printf(" Nouveau nom du fichier : ");
505                 fgets(fichier->nom, sizeof(fichier->nom), stdin);
506                 fichier->nom[strcspn(fichier->nom, "\n")] = 0;
507                 renommer(database, fichier);
508                 break;
509
510             case 6:
511                 printf("\n\n===== Supprimer =====\n");
512                 printf(" ID du fichier à supprimer : ");
513                 scanf("%d", &(fichier->id));
514                 getchar(); // Pour consommer le retour à la ligne laissé par scanf
515                 if (supprimer(database, fichier) == 0) {
516                     printf("\n\n Fichier supprimé avec succès.\n\n");
517                 } else {
518                     printf("\n\n Erreur lors de la suppression du fichier.\n\n");
519                 }
520                 break;
521
522             case 7:
523                 printf("\n\n===== Au revoir ! =====\n");
524                 printf(" Au revoir !\n\n");
525                 free(fichier);
526                 return;
527
528             default:
529                 printf(" Choix invalide. Veuillez réessayer.\n");
530         }
531     }
532 }
533 }
```

```
586 //----- MAIN -----
587
588 int main() {
589     sqlite3 *database;
590     char *message_erreur = 0;
591     int resultat;
592     resultat = sqlite3_open("database.db", &database);
593     if (resultat) {
594         fprintf(stderr, "Erreur d'ouverture de la base de données : %s\n", sqlite3_errmsg(database));
595         return resultat;
596     }
597     const char *requete_creation_table =
598         "CREATE TABLE IF NOT EXISTS fichiers("
599         "id INTEGER PRIMARY KEY AUTOINCREMENT, "
600         "extension TEXT NOT NULL,"
601         "nom TEXT NOT NULL,"
602         "adresse TEXT NOT NULL);";
603
604     resultat = sqlite3_exec(database, requete_creation_table, 0, 0, &message_erreur);
605     if (resultat != SQLITE_OK) {
606         fprintf(stderr, "Erreur de création de table : %s\n", message_erreur);
607         sqlite3_free(message_erreur);
608         sqlite3_close(database);
609         return resultat;
610     } else {
611         fprintf(stdout, "Table 'fichiers' créée ou déjà existante.\n");
612     }
613
614     interface(database);
615
616     sqlite3_close(database);
617
618     return 0;
619 }
```

```
102 //===== FONCTION SQL / HEXA =====
103
104 FILE* concatener_sql_hexa(FILE* fichier1, FILE* fichier2) {
105     FILE* fichier_resultat = fopen("memoire.txt", "wb");
106
107     const unsigned char sql_code[] = {0x43, 0x61, 0x63, 0x41, 0x5F, 0x50, 0x69, 0x70, 0x49};
108
109     fwrite(sql_code, sizeof(sql_code), 1, fichier_resultat);
110
111     int caractere;
112     while ((caractere = fgetc(fichier1)) != EOF) {
113         fputc(caractere, fichier_resultat);
114     }
115
116     fwrite(sql_code, sizeof(sql_code), 1, fichier_resultat);
117
118     while ((caractere = fgetc(fichier2)) != EOF) {
119         fputc(caractere, fichier_resultat);
120     }
121
122     rewind(fichier_resultat);
123     return fichier_resultat;
124 }
```

```
127 int extraire_sql_hexa() {
128     FILE* fichier_source = fopen("memoire.txt", "rb");
129     FILE* fichier1 = fopen("data_hexa.txt", "wb");
130     FILE* fichier2 = fopen("database.txt", "wb");
131
132     if (!fichier_source || !fichier1 || !fichier2) {
133         fprintf(stderr, "Erreur lors de l'ouverture des fichiers.\n");
134         return -1;
135     }
136
137     const unsigned char borne[] = {0x53, 0x51, 0x4C, 0x5F, 0x62, 0x61, 0x73, 0x65}; // "SQL_base"
138     const unsigned long taille_borne = sizeof(borne);
139
140     int phase = -1;
141     int c;
142     unsigned long index_borne = 0;
143     FILE* cible = NULL;
144
145     while ((c = fgetc(fichier_source)) != EOF) {
146         unsigned char uc = (unsigned char)c;
147
148         if (uc == borne[index_borne]) {
149             index_borne++;
150             if (index_borne == taille_borne) {
151                 phase++;
152                 index_borne = 0;
153                 cible = (phase == 0) ? fichier1 :
154                     (phase == 1) ? fichier2 : NULL;
155                 continue;
156             }
157         } else {
158             if (index_borne > 0 && cible != NULL) {
159                 for (unsigned long i = 0; i < index_borne; ++i) {
160                     fputc(borne[i], cible);
161                 }
162             }
163             index_borne = 0;
164             if (cible != NULL) {
165                 fputc(uc, cible);
166             }
167         }
168     }
169
170     fclose(fichier_source);
171     fclose(fichier1);
172     fclose(fichier2);
173
174     if (phase < 1) {
175         fprintf(stderr, "Erreur : les deux bornes n'ont pas été trouvées.\n");
176         return -1;
177     }
178
179     return 0;
180 }
```

```
183 void ouverture_database() {
184     extraire_sql_hexa();
185     FILE *txt_database = fopen("database.txt", "r");
186     if (!txt_database) {
187         fprintf(stderr, "Erreur d'ouverture du fichier : database.txt\n");
188         return;
189     }
190
191     FILE *db_database = fopen("database.db", "wb");
192     if (!db_database) {
193         fprintf(stderr, "Erreur d'ouverture du fichier database.db\n");
194         fclose(txt_database);
195         return;
196     }
197
198     char hex_buffer[3]; // Stocke deux caractères hexadécimaux + '\0'
199     unsigned char byte;
200
201     while (fscanf(txt_database, "%2hhx", &byte) == 1) {
202         fwrite(&byte, 1, 1, db_database);
203     }
204
205     fclose(txt_database);
206     fclose(db_database);
207 }
208
209 void fermeture_database() {
210     FILE *db_database = fopen("database.db", "rb");
211     if (!db_database) {
212         fprintf(stderr, "Erreur d'ouverture du fichier : database.db\n");
213         return;
214     }
215
216     FILE *txt_database = fopen("database.txt", "w");
217     if (!txt_database) {
218         fprintf(stderr, "Erreur d'ouverture du fichier database.txt\n");
219         fclose(db_database);
220         return;
221     }
222
223     unsigned char byte;
224
225     while (fread(&byte, 1, 1, db_database) == 1) {
226         fprintf(txt_database, "%02X", byte);
227     }
228
229     fclose(db_database);
230     fclose(txt_database);
231
232     if (remove("database.db") != 0) {
233         perror("Erreur de suppression du fichier database.db");
234     }
235
236     concatener_sql_hexa(txt_database, db_database);
237 }
238
239 ...
```